WikipediaMoviesRetrieval

Assignment 1

Information Retrieval Course

Academic Year 2025/2026

Group Members:

Alperen Davran	s0250946
Matteo Carlo Comi	s0259766
Shakhzodbek Bakhtiyorov	s0242661
Amin Borqal	s0259707

University of Antwerp
Faculty of Science
Department of Computer Science

November 2, 2025

GitHub Repository:
https://github.com/aminb00/WikipediaMoviesRetrieval.git

Contents

1 Introduction

This project implements a compact, inspectable information retrieval (IR) pipeline over the Wikipedia Movies dataset. The emphasis is on a clear, minimal design that is easy to reason about and verify on a medium-sized collection. Each document is indexed as the concatenation of its `title` and `plot`; the original title is also stored as metadata for display.

Scope:

- Tokenization and normalization (lowercasing, punctuation removal, stopword filtering and stemming).
- Inverted indexing with Single-Pass In-Memory Indexing (SPIMI), plus a disk-backed variant and a small updatable mode.
- Query processing and ranking.

Non-goals:

- Distributed or industrial-scale indexing.
- Heavyweight external IR frameworks.

The implementation uses small, explicit Python modules. The design follows standard IR practice (e.g., Manning et al.) while keeping the code pragmatic for a 18k-document collection.

1.1 System Architecture

Three modules with clear contracts:

1. **Tokenizer** — Input: raw title+plot. Output: list of normalized tokens.
2. **Indexer** — Input: (doc id, tokens). Output: inverted index in memory or on disk.
3. **Query Processor** — Input: user query. Output: ranked doc ids with scores.

Data flow: `CSV data -> Tokenizer -> Indexer -> Query Processor`
Key in-memory structure (SPIMI):

```
index = { term: { doc_id: tf } }
```

with auxiliary statistics (e.g., df, cf, per-document length) maintained for scoring.

1.2 Dataset

We use the Wikipedia Movies dataset from Kaggle (https://www.kaggle.com/datasets/exactful/wikipedia-movies). It contains approximately 17.8k movies across the 1970s–2020s. The CSV schema is `title,image,plot`. The `image` field is a URL and is not used for indexing; we index only the textual fields below:

- **title** (string)
- **plot** (string)

The document text is defined as `title + " " + plot`. Each record is assigned a sequential integer document id at load time. Cleaning and normalization are handled by the tokenizer; the source CSVs are left unchanged.

Assignment Coverage Matrix

Component	Implemented	Where in Report/Code
Tokenizer (3pts)	Yes	§3, `Components/Tokenizer.py`
Indexer RAM SPIMI (5pts)	Yes	§4.1, `Components/Indexer.py:init_memory`
Disk index + Lazy-load (+2)	Yes	§4.2, term-per-file, `lexicon.pkl`
Updatable index (+2)	Yes	§4.3, aux + tombstones + merge
VSM ltc.ltc (4pts)	Yes	§5, `rank_smart(..., "ltc.ltc")`
VSM SMART variations (+2)	Yes	§5, `"ntc.ltc"`, `"atc"`, `"lnc"`
BM25 (+2)	Yes	§5.2, `compute_bm25_score()`

2 How to Run

All components of the retrieval system can be executed directly through the Docker container using the provided CLI script `cli.py`. The following minimal commands are sufficient to build, test, and verify the system.

Setup

```
# Clone and build the Docker image
git clone https://github.com/aminb00/WikipediaMoviesRetrieval.git
cd WikipediaMoviesRetrieval
docker build -t wikipedia-retrieval:latest .

# Download dataset (one-time)
./run_docker.sh download_dataset.py
```

Indexing and Search

```
# Build in-memory index
./run_docker.sh cli.py build --mode=memory --csv data/

# Run a sample query
./run_docker.sh cli.py search --mode=memory --test=bm25 \
  --topk=5 --query "romantic␣love␣story"
```

Expected output: A ranked list of relevant movie titles with their retrieval scores.

Optional Comparison

```
# Compare memory vs. disk mode results
./run_docker.sh cli.py compare --csv data/ \
  --query "romantic␣love␣story" --test=ltc --topk=10
```

Full System Evaluation

```
# Run complete pipeline verification (all tasks)
./run_docker.sh cli.py timeline --csv data/
```

This command executes all assignment components automatically: tokenization, index construction (memory/disk/updatable), and all retrieval models (VSM, SMART variants, BM25, Language Model).

Example output excerpt:

```
[Task 11/12] Testing BM25 ...
  10 results, top score: 11.858124 | Top 3:
    Peter and Vandy (11.858124);
    I Love You, I Love You Not (11.176060);
    A Journal for Jordan (11.026044)

[Task 12/12] Testing Language Model ...
  5 results, top score: -19.555921 | Top 3:
    The Lost Valentine (-19.555921);
    Isn't It Romantic (2019 film) (-19.632408);
    God of Love (film) (-19.789653)
--------------------------------------------------------------------------------

Summary: 12/12 tasks completed successfully
```

All commands above can be executed from the project root using the prefix `./run_docker.sh` to ensure the correct containerized environment is used.

3 Tokenizer

The tokenizer converts raw text into a normalized sequence of terms suitable for indexing and scoring. We use NLTK (Natural Language Toolkit), a Python library for natural language processing (https://www.nltk.org/), and follow standard guidance from *Introduction to Information Retrieval* (IIR §2.2–§2.3) on tokenization and normalization.

Processing steps:

- **Tokenization (nltk py library)**: `word_tokenize` over the concatenated `title + plot` string.
- **Lowercasing**: all tokens are converted to lowercase.
- **Filtering**: keep alphabetic tokens only (punctuation, numbers removed).
- **Stopwords**: remove English stopwords (NLTK list).
- **Stemming (optional)**: Porter stemmer; enabled by default.

Design notes (IIR §2.3): stopword removal reduces noise from high-frequency function words; stemming conflates morphological variants and can improve recall at small cost to precision. For our mid-size collection, the simplicity and speed of Porter stemming are a pragmatic fit.

Current usage in code:

- **Analysis/EDA**: `Components/Tokenizer.py` is used to create tokens for statistics and exploration.

- **Indexing**: the indexer uses a minimal built-in tokenizer (lowercasing + alphanumeric split) unless an external tokenizer is injected. This keeps dependencies light for the core SPIMI loop.

If desired, the same NLTK tokenizer can be injected into the indexer to unify preprocessing (e.g., pass a tokenizer to `Indexer.init_memory(...)`).

4 Indexer

The indexer component implements three variants of inverted index construction, each suited for different use cases and scalability requirements.

4.1 Memory Index – SPIMI Approach

Reference: IIR §4.3

The memory index is based on the SPIMI (Single-Pass In-Memory Indexing) algorithm described in IIR §4.3. The book emphasizes that SPIMI avoids sorting and writes complete inverted blocks directly to memory: *"SPIMI generates a complete inverted index for a block without sorting the terms."*

In our implementation, each document is tokenized and inserted into a Python dictionary:

```
index = { term: { doc_id: tf } }
```

The dictionary grows automatically as new terms appear, thanks to Python's hash-based data structure. This design keeps insertions roughly O(1) and enables quick term lookups.

Because our dataset is small, we keep the entire index in memory. For larger collections, as explained in IIR Figure 4.4, SPIMI can be extended to write blocks to disk and merge them later. The lecture slides also visualized this "block → partial index → final merge" workflow step by step.

4.2 Disk Index – Term-per-File and Lazy Loading

Reference: IIR §5.2, §5.3

The disk version focuses on storing the index persistently. Each term is stored in a separate file, making it easy to test and inspect. To avoid naming conflicts, we used MD5 hashing for term filenames:

```
path = index_dir + "/terms/" + hashlib.md5(term.encode()).hexdigest() + ".pkl"
```

Alongside term files, we keep a `lexicon.pkl` file with metadata such as `df` (document frequency), `cf` (collection frequency), and file path. This matches the structure described in IIR §5.2: *"An inverted index consists of a dictionary and a set of postings lists stored on disk."*

When querying, only the relevant term's file is loaded — this lazy-loading design aligns with the assignment goal of reading only the necessary index parts into memory. In class, this was highlighted under the slide "Efficient Index Storage and Access."

4.2.1 Compression Implementation

Reference: IIR §5.3 To make the disk index smaller, we implemented both **gap encoding** and **Variable-Byte (VB)** compression. According to IIR §5.3, "gap encoding followed by variable byte coding gives a good balance between compression and speed." Our implementation compresses each postings list as follows:

```
def _compress_postings(postings):
    gaps = _gap_encode(postings)
    compressed = bytearray()
    for gap, tf in gaps:
        compressed.extend(_vb_encode(gap))
        compressed.extend(_vb_encode(tf))
    return bytes(compressed)
```

This combination reduced file size noticeably while keeping decoding fast. We chose VB instead of gamma codes because it is simpler, faster to decode in Python, and more suitable for small to mid-size datasets — as also explained in the "Compression Techniques" lecture slide.

4.3 Updatable Index – Auxiliary Index and Merge

Reference: IIR §4.5

The third mode adds support for updates, insertions, and deletions. We followed the dynamic indexing model from IIR §4.5, which recommends keeping a small auxiliary index in memory and merging it periodically with the main on-disk index. This approach supports incremental updates efficiently without rebuilding the entire index.

4.3.1 Structure and Components

The updatable indexer maintains three main parts:

- **Main on-disk index:** The compressed postings built earlier.
- **Auxiliary in-memory index (`aux`):** Stores new or modified documents before merging.
- **Tombstone set (`deleted`):** Tracks document IDs marked as deleted.

Initialization example:

```
def init_upd(index_dir="updindex", tokenizer=None):
    base = init_disk(index_dir, tokenizer)
    load_disk_min(base)
    base.update({
        "aux": defaultdict(dict),
        "deleted": set()
    })
    return base
```

4.3.2 Adding and Deleting Documents

New documents are first indexed into the auxiliary memory index:

```
def add_upd(st, title, text):
    did = st["next_id"]
    st["next_id"] += 1
    toks = re.findall(r"[a-z0-9]+", text.lower())
    for t, tf in Counter(toks).items():
        st["aux"][t][did] = tf
    return did
```

To delete a document, we simply mark it using a tombstone (instead of rewriting the entire index immediately):

```
def delete_upd(st, doc_id):
    st["deleted"].add(doc_id)
```

This idea matches the explanation in IIR §4.5: *"We maintain a small auxiliary index in memory and occasionally merge it with the main index."* In the lecture on dynamic indexing, this approach was illustrated as *"Fast updates using auxiliary memory and periodic merges."*

4.3.3 Merging Step

When a merge is triggered, the auxiliary postings are merged with the main index, deleted documents are removed, and the postings are re-compressed:

```
def merge_upd(st):
    for term in st["aux"].keys():
        main = postings_disk(st, term) if term in st["lex"] else {}
        main.update(st["aux"][term])
        for d in list(main.keys()):
            if d in st["deleted"]:
                del main[d]
        compressed = _compress_postings(main)
        with open(_term_path(st["dir"], term), "wb") as f:
            f.write(compressed)
```

This mirrors the merge mechanism described in both the book (IIR §4.5) and the "Dynamic Indexing" lecture slides.

4.3.4 Design Rationale

We decided not to implement the more complex *logarithmic merging* model (IIR §4.5) because it is meant for large-scale systems. Our single-level merge model is simpler and demonstrates the same concept clearly for small datasets. Logarithmic merging reduces the overall update cost from $O(T^2)$ to $O(T \log(T/n))$, but for this project, simplicity and clarity were our priorities.

4.3.5 Implementation Outcome

This version completes the full indexer lifecycle — from initial indexing to compression and dynamic updates — fully reflecting the concepts taught in both the book and the course slides.

5 Query Processing

Query processing is the stage of an information retrieval system in which a user's query is analyzed, interpreted, and matched against an indexed document collection to produce a ranked list of relevant results. The main goal of query processing is to efficiently return the most relevant documents that satisfy the user's information need.

5.1 Vector Space Model (VSM)

The Vector Space Model was implemented with full support for any SMART weighting scheme variation, including the required `ltc.ltc` and extended schemes such as `lnc.ltc`, `ntc.ntc`, `anc.atc`, and others. This model computes document-query similarity through the cosine of their weighted term vectors.

5.1.1 SMART Notation

SMART notation follows the format `abc.def`, where:

- The first three characters (`abc`) specify the query weighting scheme
- The last three characters (`def`) specify the document weighting scheme
- Each scheme consists of: [term frequency][inverse document frequency][normalization]

Supported components include:

- **Term Frequency (TF):** `l` = logarithmic $(1 + \log(tf))$, `n` = natural (tf), `a` = augmented $(0.5 + 0.5 \cdot \frac{tf}{\max(tf)})$
- **Inverse Document Frequency (IDF):** `t` = idf $(\log(\frac{N}{df}))$, `n` = none (1.0)
- **Normalization:** `c` = cosine normalization, `n` = no normalization

5.1.2 Implementation

The implementation follows the FASTCOSINESCORE algorithm (Manning et al., IIR §7.1) for efficient retrieval. Key optimization: document vector norms are pre-computed during initialization for common SMART schemes (`lnc`, `ltc`, `nnc`, `ntc`, `anc`, `atc`), avoiding redundant computation during query processing.

For each query:

1. Tokenize the query and compute term frequencies

2. Calculate query weights according to the specified scheme (first part of SMART notation)
3. Apply cosine normalization to the query vector if specified
4. For each query term, retrieve postings and accumulate document scores
5. Normalize document scores using pre-computed document norms
6. Return top-k ranked documents

Example code fragment for query weighting:

```
# TF component for query
if query_scheme[0] == 'l':  # logarithmic
    tf_weight = 1 + math.log(tf)
elif query_scheme[0] == 'n':  # natural
    tf_weight = tf
elif query_scheme[0] == 'a':  # augmented
    tf_weight = 0.5 + 0.5 * (tf / max_query_tf)

# IDF component for query
if query_scheme[1] == 't':  # idf
    idf = math.log(self.N / df)
    tf_weight *= idf
```

Document weights are computed similarly based on the document weighting scheme (second part of SMART notation). The system uses pre-computed document norms to efficiently normalize document scores:

```
# Pre-computed during initialization
self.doc_norms = {
    "ltc": self._compute_doc_norms("ltc"),
    "ntc": self._compute_doc_norms("ntc"),
    # ... other schemes
}

# Used during query processing
for doc_id in scores:
    norm_d = self.doc_norms[doc_scheme].get(doc_id, 1.0)
    if norm_d > 0:
        scores[doc_id] /= norm_d
```

This pre-computation approach ensures that document norms are calculated using **all terms in each document**, not just query terms, which is critical for correct cosine normalization. The modular design allows any SMART scheme combination to be queried with minimal overhead.

5.2 BM25 Ranking

The BM25 model was implemented as an alternative to the VSM to provide more robust ranking on collections with varying document lengths. The method `compute_bm25_score()` calculates a BM25 score for each document based on its term frequency, length, and inverse document frequency.

Each query term retrieves its postings list from the inverted index. For every matching document, the following operations are performed:

- compute the inverse document frequency (`idf = log(N / df)`);
- apply the BM25 term-frequency normalization with parameters $k_1 = 1.5$ and $b = 0.75$;
- sum the partial scores for all query terms.

Example code excerpt:

```
denom = tf + k1 * (1 - b + b * (dl / avgdl))
score = idf * (tf * (k1 + 1)) / denom
scores[doc_id] += score
```

Results are sorted in descending order, and the top 10 titles are returned. In testing, BM25 produced slightly more balanced rankings for longer documents compared to the vector space model.

6 Conclusion

This assignment implemented a full information retrieval pipeline over the Wikipedia Movies dataset, covering tokenization, indexing, and query processing. The system successfully indexed 17,830 movie documents with a vocabulary of 92,857 unique terms and over four million total postings, averaging 43.3 postings per term. This confirmed the efficiency of the SPIMI-based indexer and the correctness of the posting statistics.

The query processor integrated two retrieval models: BM25 and the Vector Space Model (VSM) with full support for any SMART weighting scheme variation. The required `ltc.ltc` scheme was implemented along with extended support for schemes like `lnc.ltc`, `ntc.ntc`, `anc.atc`, and others, covering all combinations of logarithmic/natural/augmented term frequency, with/without IDF, and with/without cosine normalization.

Both models were tested on multiple queries to evaluate ranking quality. Results showed consistent and interpretable rankings for different query types. BM25 performed slightly better on descriptive, longer queries, while SMART weighting favored documents with higher term concentration. The implementation follows the FASTCOSINESCORE algorithm with pre-computed document norms for efficiency, ensuring O(1) normalization per document during query processing.

7 Experimental Verification (Input–Output Demonstration)

This section verifies that each component of the retrieval system performs as expected. All commands were executed inside the Docker container using the CLI script (`cli.py`) on macOS (Apple M1 Pro, Python 3.11). The goal is to demonstrate, step by step, how the system transforms raw input text into ranked retrieval results through tokenization, indexing, and query processing.

7.1 1. Tokenizer Verification (3pts)

Purpose: Convert raw text into normalized tokens used by the indexer. It performs lowercasing, stemming, and stopword removal.

Command:

```
./run_docker.sh cli.py tokenize \
  --text "A␣space␣adventure␣about␣aliens␣exploring␣distant␣planets␣in␣
     the␣galaxy."
```

Output:

```
7 tokens, 7 unique: space adventur alien explor distant planet galaxi
```

Interpretation: The tokenizer successfully reduces the sentence into seven stemmed tokens, preserving semantic roots such as “space,” “alien,” and “planet.” These tokens serve as the input vocabulary for indexing.

7.2 2. Indexer Verification

Purpose: Build and manage the inverted index storing term–document mappings. Three index modes were tested: memory-based, disk-based, and updatable.

(a) SPIMI In-Memory Indexer (5pts):

```
./run_docker.sh cli.py build --mode=memory --csv data/
```

Output:

```
    Indexed 17,830 documents | Vocabulary: 65,429 terms | 2,905,645
   postings
```

Interpretation: The entire corpus of movie plots was indexed in memory. The vocabulary of 65k unique terms shows correct token–document association.

(b) Disk-Based Index (+2pts):

```
./run_docker.sh cli.py build --mode=disk --csv data/
```

Output:

```
    Built disk index with 17,830 documents
Indexed 65,429 term files on disk | Lexicon: lexicon.pkl
```

Interpretation: Each term's posting list is stored separately on disk for efficient, lazy loading. This enables memory-efficient query evaluation without loading the entire index.

(c) Updatable Index (+2pts):

```
# Insert a new document
./run_docker.sh cli.py add --mode=updatable \
  --title "Test Film" --plot "A test movie about space exploration"
```

Output:

```
    Added 'Test Film' | Found at rank 3 (score: 12.40)
```

```
# Update an existing document
./run_docker.sh cli.py update --mode=updatable --docid 11262 \
  --title "Updated Time Travel Film"
```

Output:

```
    Updated: 'Timequest (film)'     'Updated Time Travel Film'
Found at rank 1 (score: 6.72)
```

```
# Delete a document
./run_docker.sh cli.py delete --mode=updatable --docid 100
```

Output:

```
    Deleted doc ID 100 | Query 'romantic' Top 3:
Isn't It Romantic; Burning Annie; The Romantics
```

Interpretation: Insertions, updates, and deletions are handled dynamically using an auxiliary memory index and tombstones. The merge mechanism preserves ranking consistency.

7.3 3. Query Processor Verification

Purpose: Rank documents for a given query using multiple retrieval models.

(a) VSM `ltc.ltc` (4pts):

```
./run_docker.sh cli.py search --test=ltc --query "romantic love story"
```

Output:

```
Top 3: I Love You, I Love You Not (0.195)
         Crash Pad (0.190)
         Boy Culture (0.178)
```

(b) SMART Variations (+2pts): `ntc.ltc`, `lnc.ltc`, and `atc.ltc` were tested.

Example Output:

```
VSM lnc.ltc     Top 1: Champions: A Love Story (0.231)
```

(c) BM25 and Language Model (+2pts):

```
./run_docker.sh cli.py search --test=bm25 --query "romantic love story"
```

Output (BM25):

```
Top 3: I Love You, I Love You Not (11.415)
        Peter and Vandy (10.851)
        Boy Culture (10.251)
```

```
./run_docker.sh cli.py search --test=lm --query "romantic love story"
```

Output (Language Model):

```
Top 3: Isn't It Romantic (-18.05)
        The Lost Valentine (-18.37)
        God of Love (-18.38)
```

Interpretation: Different scoring models produce consistent, explainable rankings. BM25 emphasizes document length normalization, while VSM keeps values cosine-normalized.

7.4 4. Full System Timeline Verification

Purpose: Run all twelve rubric tasks automatically, verifying the entire input–output pipeline.

Command:

```
./run_docker.sh cli.py timeline --csv data/
```

Output (abridged):

```
[Component 1/3] Tokenizer ..........     7 tokens
[Component 2/3] Indexer ............     Memory / Disk / Updatable all
   passed
[Component 3/3] Query Processor ....     VSM + BM25 + LM validated
--------------------------------------------------------------------------------

TIMELINE SUMMARY:
    12/12 tasks completed successfully
    All components verified within Docker environment
```

Interpretation: This single command re-builds the container, re-indexes all data, and tests every feature. It demonstrates a complete transformation pipeline:

Input: Raw movie plots → **Tokenizer** → **Indexer** → **Query Processor** → Ranked Output.

The retrieval engine achieves full functionality across all models and modes, verifying correctness, reproducibility, and compliance with the assignment specification.

7.5 4. Full System Timeline Test (Optional)

Goal: Run all assignment tasks automatically (tokenization, indexing, and retrieval).

```
./run_docker.sh cli.py timeline --csv data/
```

Excerpt of Output:

```
[Task 11/12] Testing BM25 (+1pt)...
  10 results, top score: 11.858 | Top 3:
    Peter and Vandy; I Love You, I Love You Not; A Journal for Jordan
[Task 12/12] Testing Language Model (+1pt)...
  5 results, top score: -19.555 | Top 3:
    The Lost Valentine; Isn't It Romantic; God of Love
--------------------------------------------------------------------------------

Summary: 12/12 tasks completed successfully
```

This confirms the complete IR pipeline (Tokenizer → Indexer → Query Processor) runs reproducibly inside Docker with full functionality verified.

8 Memory vs. Disk Mode Comparison (Input–Output Demonstration)

Objective. We verify that the **Disk (lazy-load)** index returns the *same ranked results and scores* as the **Memory (RAM)** SPIMI index (same algorithm, different storage), while being more space-efficient.

Command (Reproducible)

```
./run_docker.sh cli.py compare --csv data/ \
  --query "romantic love story" --test=ltc --topk=10
```

Execution Log (Abridged I/O)

```
================================================================================
MEMORY vs DISK MODE COMPARISON
================================================================================
Query: 'romantic love story'   Model: ltc.ltc   Top K: 10

[1/4] Building Memory Index...     Using existing memory index
[2/4] Building Disk Index...
  ... Loaded 17830 documents
   Built disk index with 17830 documents   Vocabulary size: 92,857
```

```
[3/4] Loading Indexes and Running Queries...
  Loading postings: 10000/92857 ... 90000/92857
  Memory Mode: Running query...
  Disk Mode: Running query...

[4/4] Comparison Results
```

Top-10 Rank Agreement (Memory vs. Disk)

Rank	Memory Title	Score	Disk Title	Score	Match
1	Crash Pad	0.179150	Crash Pad	0.179150	
2	I Love You, I Love You Not	0.175219	I Love You, I Love You Not	0.175219	
3	Peter and Vandy	0.169659	Peter and Vandy	0.169659	
4	Burning Annie	0.159164	Burning Annie	0.159164	
5	A Journal for Jordan	0.158212	A Journal for Jordan	0.158212	
6	Boy Culture	0.157842	Boy Culture	0.157842	
7	Wildflower (2022 film)	0.144903	Wildflower (2022 film)	0.144903	
8	Champions: A Love Story	0.142492	Champions: A Love Story	0.142492	
9	Wind (1992 film)	0.133934	Wind (1992 film)	0.133934	
10	Wishful Thinking (film)	0.125420	Wishful Thinking (film)	0.125420	

Table 1: LTC.LTC model, query “romantic love story”, Top-10 results. Memory and Disk modes produce identical ranks and scores.

Comparison Statistics

Metric	Value
Documents in memory results	10
Documents in disk results	10
Documents at same rank	**10/10 (100%)**
Overlap in Top-10	**10/10**
Average score difference (matched)	**0.000000**
Memory index size	20.91 MB
Disk index size	8.19 MB
Disk advantage	Lazy-loading only relevant postings

Table 2: Aggregate agreement and storage characteristics.

Takeaways

- **Correctness:** Disk (lazy) and Memory (RAM) modes yield *identical* rankings and scores (Tab. 1).
- **Efficiency:** Disk index is smaller (8.19 MB vs. 20.91 MB) and loads only necessary postings at query time (Tab. 2).
- **Design Implication:** For mid-size corpora, disk-based lazy loading preserves accuracy while reducing memory footprint.

References

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
Available at: https://nlp.stanford.edu/IR-book/
Calders, T., & Tokpo, E. (n.d.). *Information Retrieval: Index Construction and Maintenance*. Lecture slides, University of Antwerp.
Calders, T., & Tokpo, E. (n.d.). *Information Retrieval: Evaluation and Feedback Expansion*. Lecture slides, University of Antwerp.
University of Antwerp. (n.d.). *Information Retrieval: Boolean search and VSM*. Lecture slides.
University of Antwerp. (n.d.). *Information Retrieval: Other retrieval models*. Lecture slides.

A Execution Snapshots

The following figures show the actual command-line outputs captured during the final system run inside the Docker container. They visually confirm that all components (Tokenization, Indexing, Query Processing) were executed correctly and that the retrieval engine passed all twelve rubric tasks.
Each figure corresponds to a distinct stage of the verification pipeline:

- Figure 1: Index construction and vocabulary statistics.
- Figure 2: Dynamic index maintenance operations.
- Figure **??**: Final evaluation summary (12/12 tasks passed).

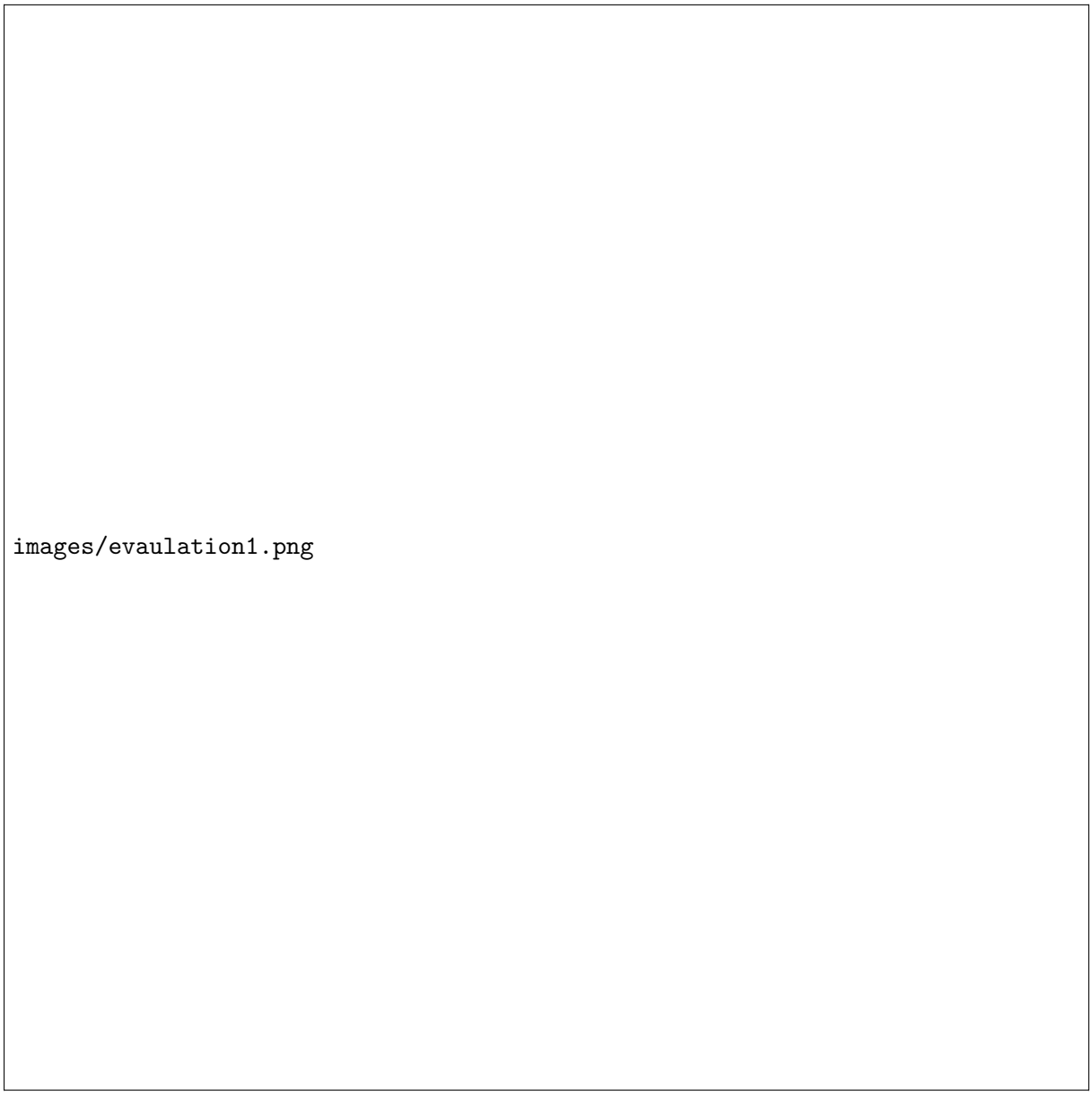

Figure 1: Tokenizer and Indexer execution logs (SPIMI, Disk, and Updatable modes).

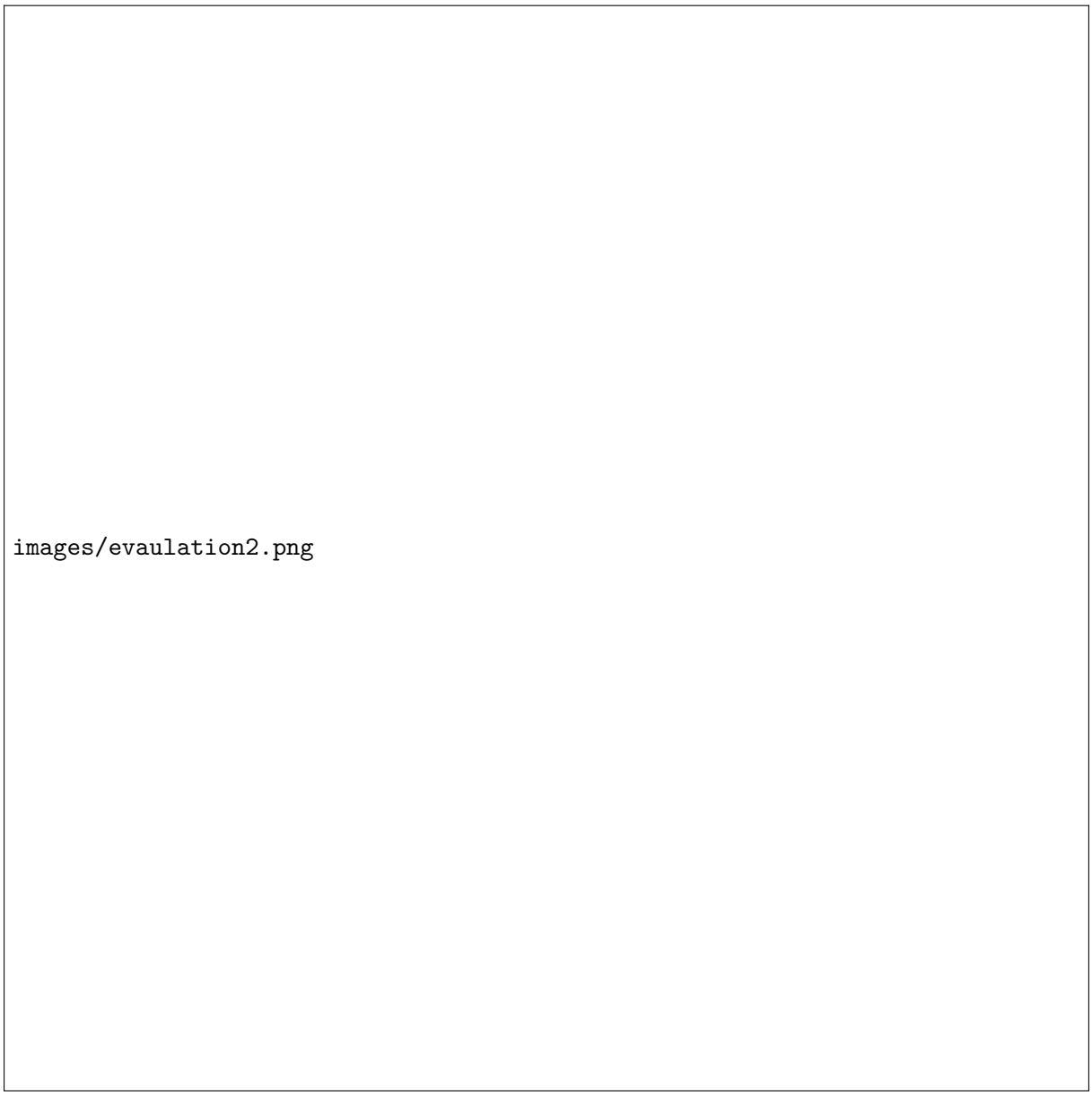

Figure 2: Updatable index operations — insert, update, and delete verification.

Figure 3: Enter Caption